CAP THEOREM (BREWER'S THEOREM)

Distributed Systems Architecture - Quick Revision Notes

1 What is CAP Theorem?

CAP Theorem (formulated by Eric Brewer) states that a distributed data store can simultaneously provide at most **two** of the following three guarantees: Consistency, Availability, and Partition Tolerance.

Core Takeaway: Because network partitions (P) are a physical reality in networked hardware, distributed systems must trade off between Consistency (C) and Availability (A) during network splits.

2 Consistency (C) Deep Dive

Linearizability / Atomic Consistency: Every read receives the most recent write or an error. All nodes see the exact same data at the exact same time, acting as if there is only a single copy of data.

Example: If a user deposits $100 into a bank account on Node A, a subsequent read on Node B must immediately reflect the updated balance, not stale pre-deposit data.

3 Availability (A) Deep Dive

Non-Error Response: Every non-failing node returns a non-error response for every request received, without guaranteeing that it contains the most recent write (stale reads are permissible).

Focus: Prioritizes system uptime and responsiveness over strict data synchronization across nodes during high traffic or degradation.

4 Partition Tolerance (P) Deep Dive

Network Partition Resilience: The system continues to operate despite an arbitrary number of message drops or delays between nodes due to network hardware failure.

Reality Check: Networks fail. Routers drop packets, cables break, datacenters lose cross-links. Thus, **P is non-negotiable** in distributed environments.

5 The CAP Trade-off: Why "CA" is a Myth

A "CA" system assumes a network can never fail. While single-node relational databases (like standalone PostgreSQL or MySQL) offer CA, they are **not distributed**.

The moment you scale horizontally across multiple servers connected via a network, partitions (P) become possible. When a partition occurs, you must choose: reject writes/fail reads (CP) or serve stale local data (AP).

PACELC & REAL-WORLD SYSTEMS

Deep Dive into Distributed System Choices and Real-World DBs

6 PACELC Theorem (Extending CAP)

Proposed by Daniel Abadi, CAP only describes behavior *during a partition*. PACELC captures normal operations: **If Partitioned (P), choose Availability (A) or Consistency (C); Else (E), choose Latency (L) or Consistency (C)**.

System Type	Partition Behavior (P)	Normal Operation Behavior (E)	Example DB
PC/EC	Consistency	Consistency (higher latency due to sync replication)	Google Spanner, HBase
PA/EL	Availability	Latency (fast async replication, risks staleness)	Cassandra, DynamoDB
PA/EC	Availability	Consistency (trade off availability during splits for low latency normal ops)	MongoDB

7 CAP in Real-World Distributed Databases

Google Spanner (CP/PC-EC): Uses TrueTime API (atomic clocks + GPS) to maintain external consistency globally across datacenters with high latency trade-offs.

Amazon DynamoDB (AP/PA-EL by default): Offers tunable consistency. Default reads are eventual (low latency, high availability), but conditional/consistent reads can be requested.

MongoDB (Configurable): Can act as CP (majority write/read concerns) or AP (local write/read concerns).

8 Summary & Cheat Sheet

- **CAP Trilemma:** Pick 2 when P occurs. You can never drop P in a network.
- **CP:** Correctness over Uptime (Banks).
- **AP:** Uptime over Correctness (Social Feeds).
- **PACELC:** Adds normal operation latency trade-offs (L vs C).

TOP SYSTEM DESIGN INTERVIEW QUESTIONS

High-Yield CAP Theorem & Distributed Trade-off Questions with Model Answers

9 Top Interview Questions & Answers

Q1: Can a database be strictly "CA"?
Answer: No. In distributed systems, network partitions (P) are inevitable due to physical network failures. A "CA" system implies a single-node database (like a standalone SQL server), which is not distributed. Once you add network links, you must account for P.

Q2: You are designing a global financial payment gateway. Do you choose CP or AP? Why?
Answer: Choose **CP**. In financial systems, allowing two partitioned datacenters to accept conflicting transactions simultaneously (AP) leads to double spending or incorrect account balances. It is better to reject transactions (unavailable) on the partitioned node than to corrupt account consistency.

Q3: How does Apache Cassandra achieve high availability (AP)?
Answer: Cassandra uses masterless peer-to-peer architecture with tunable consistency (e.g., `QUORUM`, `ONE`). During a network partition, nodes accept writes locally and use hinted handoffs and Read Repair / Anti-Entropy Merkle trees to sync data once the partition heals, prioritizing uptime.

Q4: Explain the difference between CAP and PACELC theorems.
Answer: CAP only addresses system behavior during a network partition (P). However, networks are operational 99.9% of the time. PACELC extends CAP by asking: *Else* (when the system is running normally without partitions), do you trade off **Latency (L)** or **Consistency (C)**?

Q5: Can a database be configured to switch between CP and AP dynamically?
Answer: Yes. Modern distributed databases like MongoDB or Amazon DynamoDB allow developers to tune consistency levels per query/transaction. For instance, using `WriteConcern: Majority` enforces CP behavior, while `WriteConcern: 1` leans toward AP/latency reduction.